Node JS

Learning Node JS

PRAVEEN NAIR

Top Backend Technologies

Node.js - is an open-source, cross-platform JavaScript framework that is used to build server-side and networking applications.

Django - is an open-source framework based on Python. It is a web framework from the server's side. Django follows the Model Template View (MTV) architecture.

Spring Boot - is an open-source web framework based on Java that allows developers to build production-grade and standalone applications.

ASP.NET - is an open source web framework, created by Microsoft, for building modern web apps and services that run on macOS, Linux, Windows, and Docker.

Laravel - is a PHP framework that provides a built-in user interface, flexibility, API support, creativity, and an extensive range of various libraries that help in the development process of secure web applications

Introduction to Node JS

Node.js was invented for server side scripting and uses JavaScript.

Node.js is free and open source server environment.

Node.js runs on various OS(Windows, Linux, etc.)

Node.js continues with the next request in case current request is delayed.

Node.js can collect data submitted by front end users.

Node.js can add, delete, modify data in your database

Visit https://nodejs.org/en to download and install node js

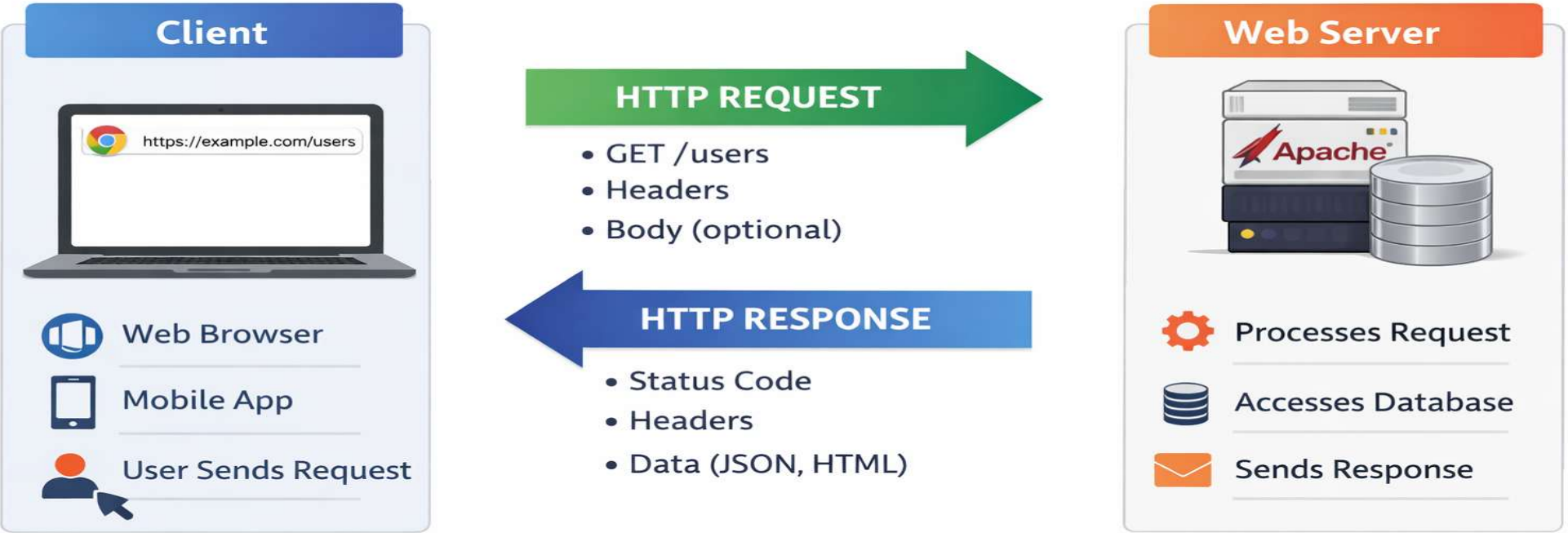
Introduction to Express JS

Express.js is fast and minimalist web framework for Node.js. An Express application is essentially a series of middleware function calls.

Express.js has powerful models for URL routing (matching an incoming URL with a server function), and handling HTTP requests and responses.

By making XML HTTP Requests (XHRs) or GETs or POSTs from your React.js front end, you can connect to Express.js functions that power your application.

Request-Response Cycle



Create and setup server

Open terminal and type the following command
npm init –y //this will create package.json file

Add following line in package.json, below name attribute
"type":"module",

Create .gitignore file (Add /node_module and .gitignore in the file)

npm install express

Installing required packages

npm install express
npm install mongodb
npm install mongoose
npm install bcrypt
npm install cors
npm install jsonwebtoken
npm install nodemon (nodemon index.js)

index1.js - creating web server

```
import express from 'express'

const app = express()

app.listen(8080)
```

.....,,,,,,,,,,,,,,,,,,,,,,

http://localhost:8080

Cannot GET /

API testing tool

Postman - One of the most popular API testing platforms.

index2.js – req/res object

```
import express from "express";
const app = express();
app.listen(8080, () => console.log("Web Server has been started"));
app.get("/", (req, res) => {
  console.log(req.url);
  console.log(req.method)
  console.log(req.body)
  res.send("response from server");
});
```

index3.js – multiple routes

```
import express from "express";
const app = express();
app.listen(8080, () => console.log("Web Server has been started"));
app.get("/", (req, res) => {
  console.log(req.url);
  res.send("response from server for route /");
});

app.get("/home", (req, res) => {
  console.log(req.url);
  res.send("response from server for route /home");
});
```

index4.js – req.params

```
app.get("/:name", (req, res) => {
  res.send(req.params.name);
});
app.get("/:name/:age", (req, res) => {
  res.send(req.params.name + req.params.age);
});
app.get("/name/:name/age/:age", (req, res) => {
  res.send(req.params.name + req.params.age);
});
```

index5.js – req.headers

```
app.get("/", (req, res) => {
  console.log(req.headers);
  console.log(req.headers.host);
  console.log(req.headers.authorization);
  res.send("response from server for route /");
});
```

index6.js – req.query

```
//http://localhost:8080/?name=amy&age=23

app.get("/", (req, res) => {
  res.send(req.query.name+req.query.age);
});
```

index7.js – res.json

```
app.get("/", (req, res) => {
  const user = {
    name: "John",
    email: "john@gmail.com",
    role: "student",
  };
  res.json(user);
});
```

index8.js – res.json – array of objects

```
const users = [
  {id:1,name: "John",email: "john@gmail.com",role: "student"},
  {id:2,name: "Admin",email: "admin@gmail.com",role: "admin"},
];
app.get("/", (req, res) => {
  res.json(users);
});
```

What is REST API?

A **REST API** (Representational State Transfer Application Programming Interface) is a way for different applications to communicate with each other over HTTP using standard web methods.

Stateless
Every request must contain all required information
Server does NOT remember previous requests.
Example:
If user logs in, client must send token with each request.

REST vs Traditional Website

Traditional Website	REST API
Returns HTML	Returns JSON
Server renders UI	Client renders UI
Full page reload	Data only

app.use

```
const logger = (req, res, next) => {
  req.msg = "Hello World";
  next();
};

app.use(logger);
app.get("/", (req, res) => {
  res.send(req.msg);
});

app.get("/", logger, (req, res) => {
  res.send("Hello " + req.msg);
});

app.get("/users", (req, res) => {
  res.send("Hello" + req.msg);
});
```

express.json – parses req.body

```
app.use(express.json()) //parses req.body to json

app.post("/", (req, res) => {

  res.json(req.body);

});
```

HTTP - Request

URL (Endpoint)

Method (get, post, put, delete, patch)

Body (for post, put and patch)

```
{
  "name": “John",
  "age": 25
}
```

HTTP - Response

Status Code

200 OK
201 Created
400 Bad Request
401 Unauthorized
404 Not Found
500 Internal Server Error

Response Body (HTML / JSON)

express.static

```
app.use(express.static("images"))

app.use("/images",express.static("images"))

app.use("/public",express.static("public"))
```

Express router

```
const userRouter = express.Router();
const postRouter = express.Router();

postRouter.get("/add", (req, res) => {
  res.send("add post");
});

userRouter.get("/add", (req, res) => {
  res.send("add user");
});

app.use("/users", userRouter);
app.use("/posts", postRouter);
```

Express router.use

```
router.use((req, res, next) => {
  console.log('Time:', Date.now())
  next()
})
```

bcrypt

```
import bcrypt from 'bcrypt'

const pwd = "pass1234"

const hashedpwd = await bcrypt.hash(pwd,10) //cost factor 12 more secure but slow

console.log(hashedpwd)

const validate = await bcrypt.compare(pwd,hashedpwd)

console.log(validate)
```

jsonwebtoken – jwt.sign

```
import jwt from "jsonwebtoken";

const SECRET = "sometext";

const user = {
  name: "john",
  email: "john@email.com",
  role: "customer",
};

const token = jwt.sign(user, SECRET, { expiresIn: "1h" });

console.log(token)
```

jsonwebtoken – jwt.verify

```
import jwt from "jsonwebtoken";

const SECRET = "sometext";

const token =

  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiam9obiIsImVtYWlsIjoiam9obkBlbWFpbC5jb
20iLCJyb2xlIjoiY3VzdG9tZXIiLCJpYXQiOjE3NTE1NjE3NjQsImV4cCI6MTc1MTU2NTM2NH0.F6ZddfX
QtDzMmVneZ_2CjSA59V6i5z7mVgaGB5S1wAM";

const user = jwt.verify(token, SECRET);

console.log(user)
```

authenticate / authorize(“admin”)

```
const authenticate = async (req, res, next) => {
const authHeader = req.headers.authorization;
if (!authHeader)
    return res.status(403).json({ message: "No token provided" });
const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = decoded;
next();
}
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.role)) {
      return res.send("Access Denied");
    } else {
      next();
    }
  };
};
app.get("/", authenticate, authorize("admin"), (req, res) => {
  res.json(users);
});
```

using mongoose

```
import mongoose from "mongoose";

mongoose.connect("mongodb://127.0.0.1:27017/temp");

const productSchema = mongoose.Schema(
  {
    name: { type: String, required: true },
    price: { type: Number, required: true },
  },

  { timestamps: true }

);

const productModel = mongoose.model("products", productSchema);
```

mongoose -list products

```
app.get("/api/products/", async (req, res) => {
  const items = await productModel.find();
  res.status(200).json(items);
});
```

mongoose - add a product

```
app.post("/api/products/", async (req, res) => {
  const { name, price } = req.body;
  const newProduct = new productModel({
    name: name,
    price: price,
  });
  await newProduct.save();
  res.status(200).json(newProduct);
});
```

mongoose - update a product

```
app.patch("/api/products/:id", async (req, res) => {
  const id = req.params.id;
  const product = req.body
  const result = await productModel.findByIdAndUpdate(id,product);
  res.status(200).json(product)
});
```

mongoose - delete a product

```
app.delete("/api/products/:id", async (req, res) => {
  const id = req.params.id;
  const result = await productModel.findByIdAndDelete(id);
  res.status(200).json(result);
});
```

Folder structure

```
backend/
  controllers/
  models/
  routes/
  middleware/
  config/

frontend/
  components/
  pages/
  context/
  services/
```

dotenv – process.env.VARIABLE

```
//.env file //
DBUSER=user1
DBPASS=pass1

//index.js//
import dotenv from 'dotenv’
dotenv.config()
const DBUSER = encodeURIComponent(process.env.DBUSER)
const DBPASS = encodeURIComponent(process.env.DBPASS)
console.log(DBUSER,DBPASS)
```

process.argv[2] – cli argument

```
import express from 'express'

const app = express()

const PORT = process.argv[2] || "8080"

app.listen(PORT,()=>{
    console.log(`Server started on ${PORT}`)
})

app.get("/",(req,res)=>{
res.send(`This app is running on PORT ${PORT}`)
})

//node index.js 8081

// node index.js 8081 3 4
//console.log(process.argv[3]+process.argv[4])
```

Deploy in Vercel

Add vercel.json file in the root of node project and pust to github

Goto Network Access in Mongodb Atlas and update ip address 0.0.0.0/0

Create account in vercel

Goto Project tab > Add New > Select github repository

Create Env > MONGODB_URI and paste the mongodb atlas string without quotes

Verson.json

```
{
"version": 2,
"builds": [
{
"src": "index.js",
"use": "@vercel/node"
}
],
"routes": [
{
"src": "/(.*)",
"dest": "index.js"
}
]
}
```

Embedded JavaScript (EJS)

```
npm install express ejs
student-crud/
├── models/
│   └── Student.js
├── routes/
│   └── studentRoutes.js
├── views/
│   ├── partials/
│   │   ├── header.ejs
│   │   └── footer.ejs
│   ├── students/
│   │   ├── index.ejs
│   │   ├── add.ejs
│   │   └── edit.ejs
│   └── layout.ejs (optional)
├── public/
│   └── style.css
├── app.js
└── package.json
```

Passing Data from Backend

```
app.get("/users", (req, res) => {
  const users = ["Amit", "Ravi", "Neha"];
  res.render("users", { users });
});
```

EJS – Basic Setup

```
const express = require("express");

const app = express();

app.set("view engine", "ejs");

app.listen(8080, () => {

  console.log("Server running on port 3000");

});
```

EJS Syntax

```
<%= name %>
.......................................
<% if (age > 18) { %>
  <p>Adult</p>
<% } else { %>
  <p>Minor</p>
<% } %>
.....................................
<ul>
  <% users.forEach(user => { %>
    <li><%= user %></li>
  <% }) %>
</ul>
```

Partials (Reusable Layouts)

```
views/
 ├── partials/
 │    ├── header.ejs
 │    └── footer.ejs
```

Express Layouts

```
npm i express-ejs-layouts
```

index.js

```
import expressLayouts from "express-ejs-layouts";
app.use(expressLayouts)
app.set("layout","layout")
.....................................................
```

views/layout.ejs

```
<body>
<header>Header Content</header>
<%- body %>
<footer>Footer Content</footer>
</body>
```

Include in Page

```
<%- include("partials/header") %>

<h1>Welcome</h1>

<%- include("partials/footer") %>
```

Form Handling with EJS

```
<form action="/submit" method="POST">

  <input type="text" name="username">

  <button>Submit</button>

</form>
.................................................
app.use(express.urlencoded({ extended: true }));  //Enable Body Parser
app.post("/submit", (req, res) => {
  res.send("Hello " + req.body.username);
});
```

Static Files (CSS / Images)

```
app.use(express.static("public"));
.................................
public/
  style.css
```